Demystifying the Lifecycle of Failures in Platform-Orchestrated Agentic Workflows

Xuyan Ma*
Institute of Software
Chinese Academy of Sciences
Beijing, China
maxuyan2021@iscas.ac.cn

Xiaofei Xie
Singapore Management University
Singapore, Singapore
xfxie@smu.edu.sg

Yawen Wang*†
Institute of Software
Chinese Academy of Sciences
Beijing, China
yawen2018@iscas.ac.cn

Junjie Wang*†
Institute of Software
Chinese Academy of Sciences
Beijing, China
junjie@iscas.ac.cn

Boyu Wu*
Institute of Software
Chinese Academy of Sciences
Beijing, China
boyu_wu2021@163.com

Mingyang Li*
Institute of Software
Chinese Academy of Sciences
Beijing, China
mingyang2017@iscas.ac.cn

Qing Wang*†
Institute of Software
Chinese Academy of Sciences
Beijing, China
wq@iscas.ac.cn

Abstract

Agentic workflows built on low-code orchestration platforms enable rapid development of multi-agent systems, but they also introduce new and poorly understood failure modes that hinder reliability and maintainability. Unlike traditional software systems, failures in agentic workflows often propagate across heterogeneous nodes through natural-language interactions, tool invocations, and dynamic control logic, making failure attribution and repair particularly challenging. In this paper, we present an empirical study of platform-orchestrated agentic workflows from a failure lifecycle perspective, with the goal of characterizing failure manifestations, identifying underlying root causes, and examining corresponding repair strategies. We present AgentFail, a dataset of 307 real-world failure cases collected from two representative agentic workflow platforms. Based on this dataset, we analyze failure patterns, root causes, and repair difficulty for various failure root causes and nodes in the workflow. Our findings reveal key failure mechanisms in agentic workflows and provide actionable guidelines for reliable failure repair, and real-world agentic workflow design.

*Also With State Key Laboratory of Complex System Modeling and Simulation Technology, Beijing, China;
Science & Technology on Integrated Information System Laboratory, Beijing, China;
University of Chinese Academy of Sciences, Beijing, China;

†Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

CCS Concepts

• **Software and its engineering → Software testing and debugging**.

Keywords

Agentic workflows, Failure lifecycle, Failure attribution, Root cause analysis, Failure repair

ACM Reference Format:
Xuyan Ma, Xiaofei Xie, Yawen Wang, Junjie Wang, Boyu Wu, Mingyang Li, and Qing Wang. 2018. Demystifying the Lifecycle of Failures in Platform-Orchestrated Agentic Workflows. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large Language Models (LLMs) have recently shown remarkable capabilities in reasoning, planning, and knowledge-intensive tasks[18], which promotes their widespread adoption across diverse application domains [4]. Building on these advances, agentic systems powered by LLMs are gaining increasing attention, as they enable multiple specialized agents to collaborate toward complex goals. Such agentic systems have been applied to software development, information retrieval and research assistance [12], where the coordination of agents often outperforms single-agent solutions.

To further lower the barrier to building such systems, a new wave of low-code agentic AI development platforms, such as Dify [6], Coze [5], n8n [20] and AutoGen [29], has emerged. These platforms provide intuitive workflow editors, pre-configured tool integrations, and flexible orchestration mechanisms, allowing users to rapidly prototype and deploy multi-agent solutions without extensive programming expertise. Although these low-code and template-based

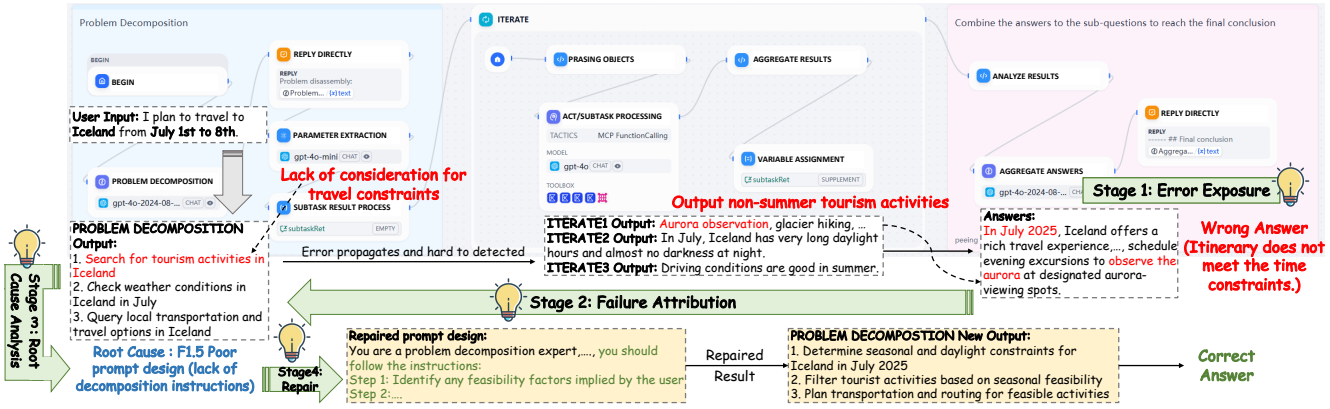

Figure 1: Example of failure lifecycle in agentic workflow.

tools have significantly lowered the bar for building agentic workflows, the inherent fragility of LLMs, such as their susceptibility to hallucinations, combined with the structural complexity of agentic orchestration makes such systems prone to failure in practice.

Similar to traditional software, repairing such failures to restore correct system behavior also requires attributing the failure, identifying its root cause, and then deriving a corresponding fix, which is a non-trivial process [28]. More than that, repairing such agentic workflows is more challenging. Unlike traditional software, which follows deterministic execution logic and well-defined control flows, real-world agentic workflows are driven by large language models and dynamic interactions. As a result, failure traces frequently exhibit rich and entangled interaction information, including natural language prompts, multi-step agent communications, and tool-invocation records [7]. Due to the inherent ambiguity of natural-language descriptions, attributing the root cause of a failure becomes highly complex. Furthermore, fixing these failures does not merely involve code modifications; it may also require refining prompts, restructuring the workflow, and other adjustments. Figure 1 illustrates an example of this process. At the final result presentation stage, the user observes that the model outputs a travel plan that does not meet the given time constraints. By carefully tracing back through 7 intricate node dependencies, systematically examining the conversational context, tool-call sequences, and natural-language reasoning chains of each agent, and cross-referencing their inputs/outputs with the intended business logic, the user eventually attributes the error: the planning agent made a mistake during task decomposition because its prompt design did not explicitly provide decomposition constraints, causing it to overlook the travel time in task decomposition. Finally, the user revises the prompt to instruct the LLM to identify any feasibility factors implied by the user, thereby fixing the issue.

Several existing studies have begun to explore failure localization and repair in LLM-based multi-agent systems. FAMAS [9] attempted to identify responsible agents and steps using spectrum analysis. AgenTracer [33] constructed TracerTraj via programmatic fault injection and counterfactual corrections, then trained a dedicated failure attributor through reinforcement learning with multi-grained rewards. Song et al. [24] designed Aegis, which employs a set of targeted environment optimizations for failure repair. Wang et al. [27] introduced Maestro, a framework-agnostic holistic optimizer that jointly searches over graphs and configurations to maximize agent quality under explicit rollout/token budgets. However, these localization and repair attempts are generally optimization-based and do not fundamentally understand the failure mechanisms or corresponding repair strategies inherent in this new paradigm of agent systems.

Cemri et al. [1] proposed MASFT (Multi-Agent System Failure Taxonomy), which identifies 14 unique failure modes involving specification issues, inter-agent misalignment, and task verification. Lu et al. [16] analyzed agent failures from the perspective of execution stages: task planning, task execution, and response generation. Zhu et al. [35] proposed AgentErrorTaxonomy, a failure modes spanning memory, reflection, planning, action, and system-level operations. Yet, these works largely describe failure manifestations without delving into their underlying root causes, let alone proposing concrete repair strategies. For example, in the MASFT taxonomy [1], the most frequent failure is "step repetition", which is merely a surface-level symptom. In Lu et al.[16], we can know the manifestations of failure "Infinite loop with same response", but still find it difficult to grasp the underlying reasons for the failure. Even when developers are informed of this failure, they cannot link it back to a root cause, which leaves them without actionable guidance for repair. This lack of holistic understanding fundamentally hinders progress in achieving reliable workflow repair.

To bridge this gap, we present an empirical study from the perspective of the failure lifecycle in platform-orchestrated agentic workflows, aiming to characterize failure manifestations, identify potential root causes, and explore corresponding repair strategies. We first introduce a dataset named AgentFail[1] that explicitly captures the entire failure lifecycle to support systematic analysis. It consists of 307 failure trajectories collected from two representative agentic workflow development platforms, annotated through multi-round expert labeling with reliable consensus. Each instance includes the user query, the agentic workflow configuration, execution failure logs, and detailed lifecycle annotations covering failure location, root cause, and repair strategy. Based on this dataset, we

[1]Website URL: https://anonymous.4open.science/r/AgentFail-F8BD

conduct detailed qualitative analysis, develop automated technique for repair, and evaluate the performance of different failure types in terms of repair success rate and cost. Moreover, we provide in-depth examination of failure occurrence patterns, root causes, and repair difficulty across various types of failure root causes and agent nodes. Together, these analyses offer a clearer understanding of the failure mechanisms in this emerging paradigm of agentic systems and deliver actionable guidelines to facilitate the development of more reliable and robust automated failure attribution and repair approaches. Besides, we also provide practical guidelines for developers to construct more robust agentic workflows on platform, ensuring that our study not only deepens understanding but also supports real-world platform practices.

2 Background

Agentic AI Development Platform. Modern agentic AI development platforms (e.g., Dify, Coze, n8n) expose agentic workflow construction through a node-based paradigm, where each node corresponds to a functional unit. By connecting nodes, users can define complex multi-agent system that combine reasoning, control, and external interactions. The **node types** are summarized as follows: 1) Start and Termination Nodes; 2) LLM and Agent Nodes; 3) Knowledge Nodes; 4) Logic and Control Nodes; 5) Code and Template Nodes; 6) Tool and Integration Nodes. These nodes extend the platform's capability beyond built-in components, enabling workflows to interact with external services, databases, or custom tools. These node types are further analyzed in Sec. 4.1 except Start and Termination nodes because they serve as the entry and exit markers of the workflow and do not directly contribute to execution failures.

Failure Definition. Since we would like to collect failure trajectories of agentic workflows, we first need to determine whether the current execution has deviated from expected behavior. In our setting, the execution of system interrupts is directly regarded as a failure. For the concluded executions, we adopt the following two evaluation strategies for different types of tasks:

Ground Truth Comparison: For tasks with deterministic ground truth or validation mechanism (e.g., code generation), we directly compare the outputs against the provided ground truth. One execution is considered successful if the output either passes test-based verification (e.g., functional correctness in code execution) or exactly matches the reference solution.

Multi-LLM Judge: For tasks lacking deterministic ground truth (e.g., task planning, deep research), we adopt LLM-as-a-judge [11] technique. Specifically, we use multiple independent LLMs as evaluators, each tasked with assessing whether the system output satisfied the task requirements. Then, a consensus voting strategy is applied, where an output was marked as correct if a majority of LLM judges agreed on its validity.

To identify the root location of failure where the failure originates, we consider node actions that could be decisive: if replacing a wrong action with a correct one (while keeping previous steps unchanged) changes the trace from failure to success, we regard this action as decisive. When multiple such actions exist, we follow an earliest-in-time principle and select the first decisive action as the failure root. Based on the identified location, we further analyze and summarize the underlying root causes of failure.

3 The AgentFail Dataset

We collect systems execution data from two popular agentic AI development platforms, Dify and Coze, which provide the capabilities of visual workflow composition and tool integration. The dataset AgentFail contains 307 failure logs from ten platform-orchestrated agentic workflows in total, with five from each platform. Each instance in the dataset includes four elements: (1) Query, a query obtained from the real test case; (2) Failure log: the full conversational trace of a system failing to complete the task. (3) Workflow and configuration: including the node orchestration structure and the information of each node, like agent's name, agent's prompt, code, tool configuration. (4) Annotations: the root location of failure, the root cause taxonomy labels, further explanation about why the failure took place and the corresponding repair strategy.

Table 1: Dataset Information. For test data sources with a large scale, we randomly select a subset of 100 samples to form the test set. We use the format `Hand-crafted-Platform Name-Task Name` to name the test data created by ourselves. The failure logs come from our run or community.

Platform	Task	Structure	Test Data Source	Test Data size	Failure Log (Run + Community)
Dify	Code Generation	Looping	HumanEval [3]	163	23 (21+2)
	Program Repair	Looping	SWE-bench [14]	100	62 (62+0)
	Product QA Assistant	Branching	Hand-crafted-Dify-QA	100	18 (17+1)
	Travel Assistant	Parallel	TravelPlanner [31]	180	32 (31+1)
	Deep Research	Hybrid	ninja-x-deepreasearch [21]	182	27 (26+1)
Coze	Product QA Assistant	Hybrid	Hand-crafted-Coze-QA	100	37 (34+3)
	Travel Assistant	Parallel	TravelPlanner	180	31 (28+3)
	Market Research Assistant	Serial	Hand-crafted-Dify-Market	100	30 (29+1)
	Deep Research	Serial	ninja-x-deepreasearch	182	27 (25+2)
	Industry Analysis	Parallel	Hand-crafted-Dify-Industry	100	20 (20+0)
Sum				**1,387**	**307 (293+14)**

3.1 System selection

The agentic workflows span a variety of task categories, including software development, information insight, task planning, and question & answering. Both platform-provided templates and user-defined systems we select from the platforms are included to ensure coverage of typical cases. To ensure diversity, we consider not only the task types but also the structural design of systems, covering five common categories: serial, parallel, branching, looping, and hybrid structures, whose details can be found on the website. This choice enables our analysis to cover diversified failure modes that are specific to both task semantics and orchestration patterns.

3.2 Failure log collection

Failure logs analyzed in this study are obtained from two sources: (1) open-source community contributions, where users had publicly shared, whose example can be found on the website. From these reports, we reproduce the executions based on the user input to construct failure cases; and (2) our own controlled runs, in which we execute public datasets (e.g., HumanEval [3], TravelPlanner [31]) or hand-crafted datasets based on the task, 1,387 test data in total, on corresponding workflow and systematically record the traces. For hand-crafted datasets (denoted as Hand-crafted-Platform name-Task name), we carefully design the construction criteria to ensure the representativeness and diversity of the inputs, which can be found on our website. For each run, success or failure is determined by the criteria described in Sec. 2. Finally, we collect 307 failure logs totally and these combined sources ensure that our dataset captured both naturally occurring failures and reproducible benchmark failures. Detailed stats of AgentFail are listed in Table 1.

3.3 Failure Root Location & Cause Annotation

To systematically identify both where failures originate and why they occur, we identify the failure root location and root cause based on multiple rounds of annotation. During the process, we adopt Grounded Theory (GT) annotation method [10], which is a qualitative research method that directly constructs theories from empirical data, to identify root cause patterns.

(i) **Independent Annotation**. Three annotators with expertise in agentic systems independently examined the execution traces, where each annotator manually identifies the decisive errors and corresponding root cause based on the expert knowledge and understanding of failure logs. Additionally, each expert is instructed to separate their annotations into two categories: cases where they are fully confident in the identified failure, and cases involving any degree of uncertainty. (ii) **Consensus Building**. Annotators then focus specifically on the cases marked as uncertain in the first step. These instances are jointly reviewed, and through collaborative discussion, agreement is reached on the final labels. (iii) **Cross Validation**. Finally, we adopt a cross-validation procedure. Each expert reviews the annotations made by others to assess the consistency of labeling standards. If any discrepancies are identified, the annotators engage in further discussion and, when necessary, re-annotate the data together until consensus is achieved. By incorporating multiple perspectives and enforcing agreement among annotators, this process enhances the reliability of the final annotations.

4 Demystifying Failure Lifecycle

4.1 Failure Attribution

4.1.1 ***Failure Manifestation***. We conduct a comprehensive analysis of all failure attribution nodes in the dataset. Specifically, we first examine the types of nodes that are identified as being responsible for the final failure, as defined in Sec. 2. Secondly, we also analyze their positions within the workflow execution, including the distance to the point where failures are ultimately exposed and their distances relative to the start and end of the workflow.

Node Type. We present the types of nodes that are more likely to introduce failures in AgentFail to demystify the failure distribution. As Figure 2(a) shows, LLM & Agent nodes dominate failure root causes, accounting for over half of all cases, indicating that agent-level reasoning and planning are the primary sources of failures in agentic workflows. The logic & control nodes and knowledge nodes are the second and third most error-prone nodes respectively, suggesting that unreasonable orchestration and wrong knowledge retrieval also play significant roles. Code and template nodes, whose failures are related to traditional software, account for only a very small proportion of all failures cases, highlighting that in this emerging agent-based systems, the failure patterns and corresponding repair priorities are fundamentally different from those in traditional software systems.

Position. We further analyze the failure root position by measuring the distance $\mathbb{D}$ between the root cause node and the failure exposed node, to characterize how long an error propagates through the workflow before becoming observable. Since agentic workflows vary in length, we normalize this distance $\mathbb{D}$ by dividing its value by the total length of the workflow, enabling fair comparison across workflows of different scales.

Figure 2(b) illustrates the distribution of the normalized distance across all failed workflows. In 32% of failures, the root cause node differs from the failure symptom node, with propagation distances exceeding 40% of the workflow length in over 10% of cases. Particularly, nodes such as LLM&Agent and logic & control are prone to such long-distance error propagation, significantly amplifying the challenge of failure repair. As Figure 2(c) shows, for LLM & Agent nodes, approximately 40% of failure root causes occur at locations different from where the failures are exposed. This proportion is even higher for Logic & Control nodes, reaching 45%, indicating that failures originating from these nodes are more likely to propagate across multiple nodes and manifest as non-local failures. Due to the space limitation, we present the remaining analysis result on the website, including the distance between root cause nodes and

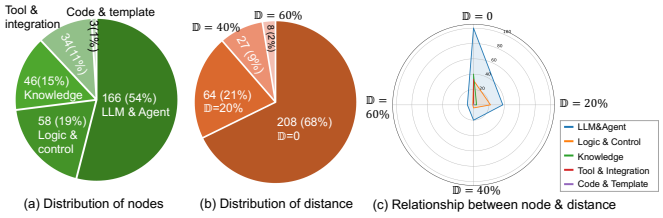

Figure 2: Statistic of Distribution. In (c), each concentric ring indicates the number of failure cases.

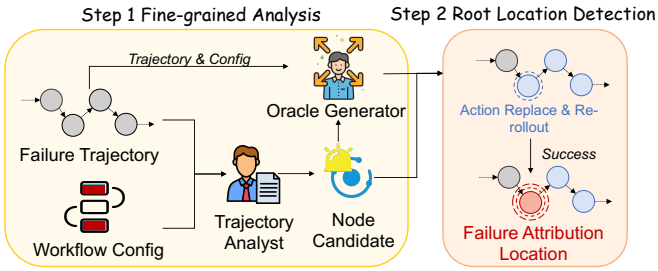

Figure 3: The process of failure attribution.

the start node, as well the termination node. These analyses help characterize how early failures are introduced during workflow execution and to what extent their effects propagate, providing further insights into the non-local nature of failures in agentic workflows.

4.1.2 ***Automatic Failure Attribution Performance.*** As mentioned in Sec.1, failure attribution is challenging and often requires substantial manual effort. Therefore, we aim to investigate the performance of automated failure attribution for agentic workflows. Following prior work [35], we customize an automated framework for failure attribution tailored to agentic workflows, as Figure 3 shows. It consists of two key stages: in Step 1 (Fine-grained Analysis), we jointly analyze the failure trajectory and the workflow configuration to systematically examine agent interactions, control flows of workflow, and intermediate outputs, and identify a small set of candidate suspicious nodes that are likely to be responsible for the observed failure. This effectively narrowed down the search scope for failure attribution. In Step 2 (Root Location Detection), we perform re-rollout-based counterfactual verification on the candidate nodes identified in Step 1. Specifically, we selectively replace the outputs of a candidate node with corrected ones (generated by Oracle Generator) while keeping the rest of the execution unchanged, and re-execute the workflow to observe whether the failure is resolved. The earliest node whose correction turns the failure into a success is identified as the root failure location. Together, these two steps transform failure attribution from heuristic inspection into a principled, causality-oriented process.

As shown in Table 2, failure attribution without re-rollout verification (i.e., w/o RLDetect) achieves 46.3% accuracy. In contrast, incorporating the re-rollout-based root location detection (illustrated in Figure 3) improves the attribution accuracy to 65.8%. The result indicates that failures in agentic workflows often involve cross-node dependencies and long-range failure propagation, which makes one-shot, end-to-end attribution vulnerable to being misled by local evidence, noisy execution logs, or surface-level symptoms. Without explicit validation, attribution models may incorrectly assign responsibility to nodes that are temporally or structurally close to the error manifestation but not causally responsible.

4.2 Failure Root Cause

4.2.1 ***Taxonomy & Distribution.*** We systematically analyze the failure cases observed in agentic workflows and construct a structured, repair-oriented failure root cause taxonomy, as shown in Figure 4. The leftmost layer classifies failures according to the system abstraction level at which they originate. The middle layer specifies concrete failure root causes, while the rightmost layer summarizes corresponding repair strategies. Agent-level failures capture failures that occur within a single agent, primarily due to limitations of the underlying LLM or its interaction with local resources; Structure-level failures arise from coordination or communication among other agents, often linked to workflow orchestration structures; Platform-level failures are attributed to the underlying platform or runtime environment. As mentioned in Sec.1, many existing taxonomies primarily characterize failure manifestations, such as "step repetition" in MASFT[1] and "infinite loop with same response" in Lu et al. [16], which neither aid in understanding the root causes of failures nor offer guidance for repair. However, our taxonomy is repair-oriented: each category corresponds to a fundamental cause of failure and is mapped to concrete repair strategies, thereby directly supporting actionable fixes.

Figure 4 also demonstrates the occurrences of each root cause. Overall, agent-level failures dominate the dataset, with knowledge and reasoning limitations (F1.4) and poor prompt design (F1.5) being the most frequent categories. Structure-level failures are less common but remain important, with missing input validation (F2.1) and unreasonable node dependency (F2.2) appearing most often, while structural issues such as loops, deadlocks (F2.3), and improper task decomposition (F2.5) are relatively rare. Platform-level failures constitute the smallest proportion, including network or resource fluctuations (F3.1) and service unavailability (F3.2). Due to space limitations, additional data analyses are provided on our website.

4.2.2 ***Automatic Root Cause Identification Performance.*** We further explore how automated techniques can identify the root cause, and investigate whether failure attribution and our proposed root cause taxonomy can assist this process. Specifically, we prompt the LLM to act as a root cause analyst. Given the failure attribution node, together with the execution trajectory and workflow configuration, we leverage our proposed root cause taxonomy to guide the analyst toward structured and systematic reasoning about the failure. In our experiments, we consider four different settings by varying whether the failure attribution location and the root cause taxonomy are provided.

Table 2 shows that providing both root cause taxonomy and failure location substantially facilitates root cause identification. When neither taxonomy nor failure location is available, the model achieves only 9.6% accuracy, indicating that root cause identification in a fully unguided setting is extremely challenging. Introducing the root cause taxonomy alone leads to a significant performance improvement, boosting accuracy to 27.4%. This improvement suggests that taxonomy provides strong conceptual guidance by constraining the model's reasoning space to semantically meaningful failure categories, enabling more structured causal inference even without explicit localization information. However, the gains from location alone are consistently smaller than those achieved by taxonomy alone. This highlights that root cause identification is not merely a localization problem, but fundamentally a semantic reasoning task. While failure location answers where the failure is observed, it does not directly explain why the failure occurs, leaving substantial ambiguity in complex agentic workflows.

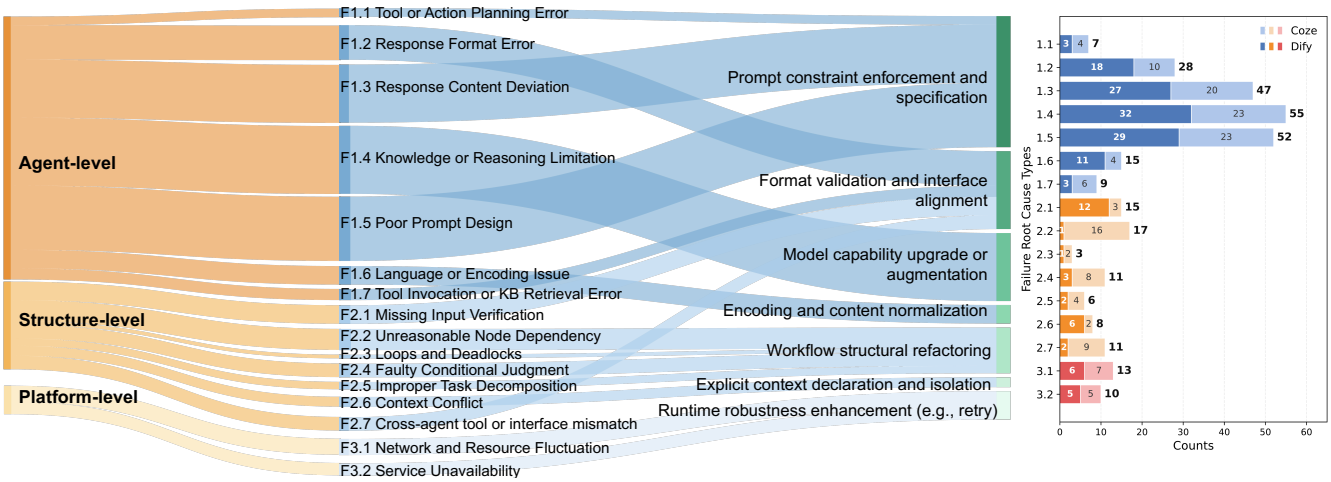

Figure 4: Failure Root Cause and Repair Strategy Taxonomy.

Table 2: Performance comparison of failure attribution, root cause identification. RLDetect represents the root location detection in Figure 3. LO and TA respectively represent failure location and root cause taxonomy.

Failure Attribution		Root Cause Identification			
w/o RLDetect	with RLDetect	w/o LO, w/o TA	w/o LO, with TA	with LO, w/o TA	with LO, with TA
46.3%	65.8%	9.6%	27.4%	22.8%	45.6%

The best performance is achieved when both taxonomy and failure location are provided simultaneously, with accuracy reaching 45.6%. This synergy indicates that taxonomy and localization play complementary roles. Taxonomy offers an abstract causal framework that defines the space of plausible failure mechanisms, while failure location grounds this reasoning in concrete execution traces. Together, they jointly constrain both the semantic and structural dimensions of the problem, enabling more precise and stable root cause identification. Meanwhile, the 45.6% accuracy rate also indicates that identifying the root cause is a very challenging task.

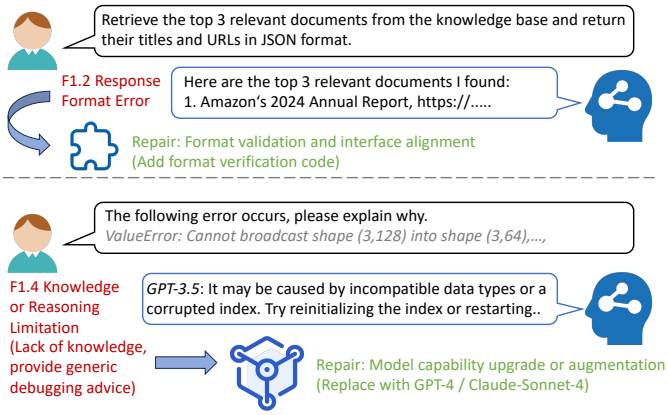

Figure 5: Examples of failure and repair.

4.3 Failure Repair

4.3.1 ***Repair Strategy Taxonomy.*** We systematically study what are the repair strategies for repairing failed agentic workflows. By analyzing how specific failure causes manifest in agentic workflows, we identify the actionable strategies that directly target the underlying issues, as shown in Figure 4. Figure 5 illustrates two failure modes and the corresponding repair method in agentic workflows: response format errors (F1.2) and knowledge or reasoning limitations (F1.4). The first failure arises from misaligned output formats, which can be repaired via adding format verification module; the second stems from insufficient model capability, requiring upgrades or external augmentation for effective comprehension and debugging.

4.3.2 ***Automatic Repair Performance.*** We construct a repair framework based on popular agent-based methods in traditional program repair domain. Drawing inspiration from established methods such as SWE-agent [32] and Agentless [30], we construct a failure repair team consisting of four specialized experts, who collaboratively perform the repair task in a staged manner, with each expert responsible for a distinct phase of the repair process. During this process, the experts leverage failure trajectories, workflow configurations, as well as explicit information about failure locations and root causes (if any) to guide their decisions and actions. This setting enables us to examine how failure attribution and root cause knowledge contribute to effective and reliable workflow repair. Accordingly, we apply two other experimental settings based on whether to use failure attribution and root cause information to

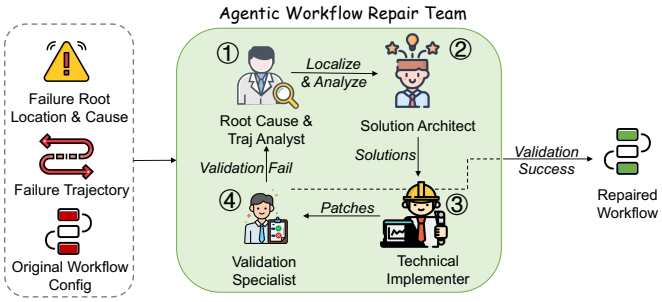

Figure 6: The process of repair.

explore the significance of the two components. Notably, the setting "w/o LO & with RC" is not included since it is an unreasonable experimental setup.

Firstly, we investigate the **repair success rate and whether the act of repairing introduces any new failures**. The new failure rate is measured by re-running the repaired workflow on the original set of test inputs that previously passed (1387 − 307) and computing the proportion of failed cases. As shown in Table 3, as the repair guidance information gradually becomes more comprehensive, the repair success rate significantly increases and the new error rate continuously decreases. Especially when both the failure root location and the root cause information are provided simultaneously, the repair success rate reaches 66.8%, and the new error rate is only 2.8%, demonstrating a more effective and safer repair outcome. When neither the failure location information nor the root cause information is provided, a large number of new errors are introduced, indicating that the repair behavior is unreliable.

Secondly, we analyze **repair performance across different root cause categories** by measuring repair success rates and the number of repair iterations. As shown in Figure 7, root causes located closer to the lower right region of the figure correspond to higher repair success rates and lower repair costs, and are therefore easier to repair. We can find that agent-level root causes are clustered in the region with high repair success rates and low iteration costs, indicating that they are relatively easy to repair through targeted modifications. In contrast, structure-level root causes, particularly those involving control logic, node dependencies, or task decomposition, exhibit lower repair success rates and require more repair iterations. F2.1 (Missing input verification) is an exception: although it is related to workflow orchestration, it can often be repaired by simply adding input validation mechanisms, resulting in a relatively high repair success rate. Moreover, we analyze the sensitivity of repairing different root causes to the availability of failure localization and root cause information. We find that the causes

Table 3: Performance comparison on failure repair. LO and RC respectively represent the failure location and root cause.

Methods	Repair Success Rate	New Failure Rate
w/o LO & w/o RC	16.3%	13.9%
with LO & w/o RC	34.9%	8.4%
with LO & with RC	**66.8%**	**2.8%**

related to formatting (F1.2), encoding (F1.6) have a relatively low sensitivity to the failure location and root cause information. Even in the absence of clear guidance, they can often be repaired. We also observe that underlying causes that require deeper semantic understanding (e.g., F1.5 Poor Prompt Design) exhibit a substantial decrease in repair performance when root cause information is absent. Moreover, when the failure root location is further removed, the repair rate continues to drop.

Furthermore, we investigate the **repair rates across different node types**, revealing a clear relationship between node characteristics and repair effectiveness, as shown in Figure 7. We can see that nodes with more deterministic structures, such as code and template nodes, exhibit consistently higher repair success, whereas LLM & Agent nodes and tool & integration nodes remain more difficult to repair. The observed differences in repair rates suggest that workflow developers should explicitly account for node heterogeneity during design. In particular, failures originating from LLM & Agent nodes and tool and integration nodes are harder to repair, indicating that these nodes should be treated as high-risk components. For more deterministic components, such as code, template, and knowledge nodes, developers can rely on stricter specifications and automated checks, as errors in these nodes are more likely to be localized and fully repairable.

5 Discussion

In order to design more effective automatic failure repair methods, we further analyze the cases that are not repaired and identify three main reasons: (1) the bottleneck of the repair agent's capabilities, making it difficult to obtain reliable correct answers, which often leads to incorrect repair directions; (2) the inability to break out of cognitive boundaries, which result in very few or no attempts at repair; (3) platform mechanism restrictions, which make it difficult for ordinary developers to avoid current failures. Based on these observations, we derive the following recommendations for future automated failure repair systems. (i) First, for collaborative repair teams, it is encouraged for them to **actively challenge existing assumptions and explore different viewpoints**. In this way, they might achieve this goal by constructing broader or opposing test scenarios. This helps to overcome cognitive biases and enables the repair work to go beyond the current reasoning scope of the team. (ii) Secondly, **incorporating memory mechanisms or template-based repair strategies** can be beneficial, as repair agents may have encountered similar workflow failures in the past, yet such experiences are often underutilized, preventing effective knowledge transfer across repair attempts.

Our ultimate goal is to enable developers to create more robust agentic workflows, so we summarize the following actionable guidelines to help improve the process of developing workflows: (i) **Clear role specification and modular prompt design** can mitigate planning errors and response misalignments, two of the most frequent failure root cause categories we observed (related with F1.3 Response content deviation, F1.5 Poor Prompt design). (ii) **Explicit input and output validation** should be incorporated into nodes to prevent cascading errors from malformed data (related with F1.2 Response formatting error, F2.1 Missing input validation). (iii) **Comprehensive checks or fallback mechanisms**, such as

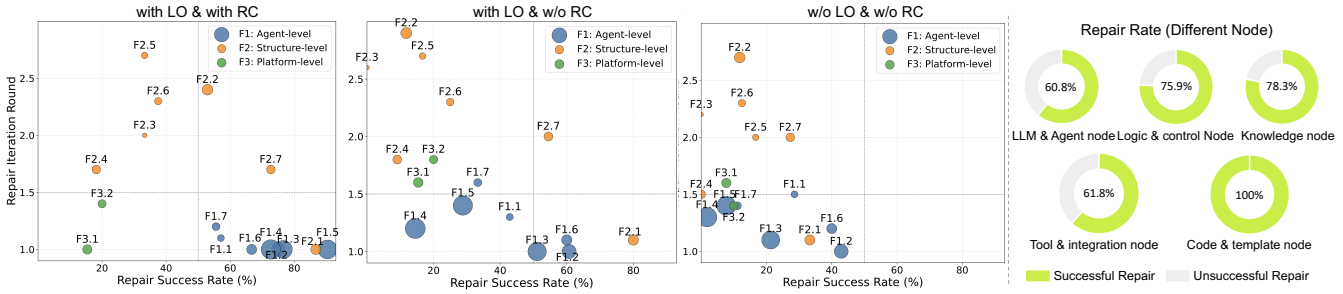

Figure 7: Repair Difficulty of Different Root Cause and the repair rate of different node. (In first three subfigures, the size of each point represents the number of failures labeled as that root cause.)

secondary validation agents or alternative tool paths, help solve the problems at the local level before they propagate across the workflow (related with F2.4 faulty conditional judgment). Sometimes, such comprehensive checks may introduce additional overhead, which requires a balance between robustness and efficiency. (iv) **Progressive workflow design**, which starts with simple serial or parallel flows and gradually introduces complex patterns. It can help build more robust systems (related with F2.2 unreasonable node dependency, F2.3 loops and deadlock).

6 Related Work

LLM Multi-Agent Systems. Recent work has explored LLM-based MAS across a variety of application settings. A representative line of research focuses on agents for software engineering tasks, such as code generation, and program repair, exemplified by systems like AutoGen [29], ChatDev [22], MetaGPT[13], AutoAgents[2] and SWE-agent [32]. For example, MetaGPT [13] organizes LLM agents into structured roles that follow software engineering workflows to collaboratively generate complex software artifacts from high-level requirements. SWE-agent [32] focuses on autonomously program repair through iterative interaction with code repositories, tools, and execution environments. In addition, there are also some agents designed for other types of tasks. Both Magentic-One[8] and Captain Agent[25] are capable of executing tasks from the GAIA[19] benchmark. Magentic-One employs a multi-agent system to collaboratively solve GAIA tasks, while Captain Agent leverages an automatically constructed agent team to perform task decomposition and execution, enabling effective problem solving across diverse GAIA scenarios. CAMEL[15] introduces a role-playing communication framework that guides chat-based language agents to cooperate autonomously via prompt-driven interactions. The agentic workflows studied in this paper are built on low-code platforms. However, they form a distinct subclass characterized by visual, template-driven configurations rather than pure code, which introduces unique challenges in workflow design, debugging, and repair.

Failure Analysis for agentic systems. As LLM-based MASs grow increasingly complex, failures in agent coordination [17, 26], tool usage [23], and workflow execution have become a prominent challenge. Recent studies have begun to explore failure localization, diagnosis, and repair in such systems. Some works focus on failure localization and responsibility attribution. Zhang et al. [34] constructed Who&When dataset to study agent- and step-level failure attribution, while Ge et al. [9] proposed methods to estimate agent responsibility in failed executions. Zhang et al. [33] investigated fine-grained failure localization using curated trajectories. Beyond failure localization, some efforts explored failure repair and optimization, such as Aegis [24] and Maestro [27], which aim to improve system robustness through automated optimization. Another line of work seeks to analyze the root cause of agent failures. Cemri et al. [1] proposed the MAST taxonomy to summarize common failure patterns in multi-agent systems. Lu et al. [16] and Zhu et al. [35] analyzed failures from the perspectives of execution stages and agent capabilities, respectively. However, existing studies primarily focus on where failures occur or how they manifest, rather than why they arise and how to repair them. Many identified failure types remain at the level of surface phenomena and do not directly translate into actionable repair guidance. Our work addresses this limitation by emphasizing root-cause–level analysis and proposing a structured taxonomy that bridges failure manifestations and systematic workflow repair.

7 Conclusion

Unlike traditional software, failure traces in emerging agentic workflows contain complex interaction information, including natural language prompts, multi-agent communications, and tool invocation logs, making their repair inherently more challenging. This paper presents a systematic investigation of the whole failure lifecycle in platform-orchestrated agentic workflows. Through the construction of AgentFail, a dataset of 307 annotated failure traces, we enable fine-grained analysis of where failures originate, how they propagate, and how to repair them. We introduce a repair-oriented root cause taxonomy that bridges the gap between failure manifestations and actionable repair strategies. In addition, we conduct an in-depth investigation into different types of failure root causes and nodes, examining their failure occurrence patterns, underlying causes, and repair difficulty. These analyses together provide a clearer understanding of failure mechanisms in this emerging agent-based system paradigm and yield actionable insights to guide the development of more reliable and robust automated failure attribution and repair methods. Our work also offers practical design guidelines for building more robust agentic workflows, emphasizing modular prompt design, explicit validation, comprehensive check, and progressive design process.

References

[1] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. 2025. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657* (2025).

[2] Guangyao Chen, Siwei Dong, Yu Shu, Ge Zhang, Jaward Sesay, Börje F. Karlsson, Jie Fu, and Yemin Shi. 2024. AutoAgents: A Framework for Automatic Agent Generation. arXiv:2309.17288 [cs.AI] https://arxiv.org/abs/2309.17288

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] https://arxiv.org/abs/2107.03374

[4] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2023. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. arXiv:2308.10848 [cs.CL] https://arxiv.org/abs/2308.10848

[5] Coze Contributors. 2025. Coze. https://www.coze.com/.

[6] Dify Contributors. 2023. Dify. https://dify.ai.

[7] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang (Eric) Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM, 1–15. doi:10.1145/3706598.3713581

[8] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang, Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. 2024. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. arXiv:2411.04468 [cs.AI] https://arxiv.org/abs/2411.04468

[9] Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. 2025. Who is Introducing the Failure? Automatically Attributing Failures of Multi-Agent Systems via Spectrum Analysis. arXiv:2509.13782 [cs.SE] https://arxiv.org/abs/2509.13782

[10] B. Glaser and A. L. Strauss. 1968. The Discovery of Grounded Theory: Strategy for Qualitative Research. *Nursing Research* 17, 4 (1968), 377–380.

[11] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594* (2024).

[12] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. arXiv:2402.01680 [cs.CL] https://arxiv.org/abs/2402.01680

[13] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352 [cs.AI] https://arxiv.org/abs/2308.00352

[14] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).

[15] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Thirty-seventh Conference on Neural Information Processing Systems*.

[16] Ruofan Lu, Yichen Li, and Yintong Huo. 2025. Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks. arXiv:2508.13143 [cs.AI] https://arxiv.org/abs/2508.13143

[17] Samuele Marro, Emanuele La Malfa, Jesse Wright, Guohao Li, Nigel Shadbolt, Michael Wooldridge, and Philip Torr. 2024. A Scalable Communication Protocol for Networks of Large Language Models. arXiv:2410.11905 [cs.AI] https://arxiv.org/abs/2410.11905

[18] Andrea Matarazzo and Riccardo Torlone. 2025. A Survey on Large Language Models with some Insights on their Capabilities and Limitations. arXiv:2501.04040 [cs.CL] https://arxiv.org/abs/2501.04040

[19] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: a benchmark for General AI Assistants. arXiv:2311.12983 [cs.CL] https://arxiv.org/abs/2311.12983

[20] n8n Contributors. 2019. n8n. https://n8n.io.

[21] Ninja-X Team. 2024. Ninja-X-DeepResearch. https://huggingface.co/datasets/ninja-x/deepresearch. Hugging Face dataset.

[22] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. arXiv:2307.07924 [cs.SE] https://arxiv.org/abs/2307.07924

[23] Weizhou Shen, Chenliang Li, Hongzhan Chen, Ming Yan, Xiaojun Quan, Hehong Chen, Ji Zhang, and Fei Huang. 2024. Small LLMs Are Weak Tool Learners: A Multi-LLM Agent. arXiv:2401.07324 [cs.AI] https://arxiv.org/abs/2401.07324

[24] Kevin Song, Anand Jayarajan, Yaoyao Ding, Qidong Su, Zhanda Zhu, Sihang Liu, and Gennady Pekhimenko. 2025. Aegis: Taxonomy and Optimizations for Overcoming Agent-Environment Failures in LLM Agents. *arXiv preprint arXiv:2508.19504* (2025).

[25] Linxin Song, Jiale Liu, Jieyu Zhang, Shaokun Zhang, Ao Luo, Shijian Wang, Qingyun Wu, and Chi Wang. 2025. Adaptive In-conversation Team Building for Language Model Agents. arXiv:2405.19425 [cs.CL] https://arxiv.org/abs/2405.19425

[26] Junlin Wang, Roy Xie, Shang Zhu, Jue Wang, Ben Athiwaratkun, Bhuwan Dhingra, Shuaiwen Leon Song, Ce Zhang, and James Zou. 2025. Improving Model Alignment Through Collective Intelligence of Open-Source LLMS. arXiv:2505.03059 [cs.CL] https://arxiv.org/abs/2505.03059

[27] Wenxiao Wang, Priyatham Kattakinda, and Soheil Feizi. 2025. Maestro: Joint Graph & Config Optimization for Reliable AI Agents. *arXiv preprint arXiv:2509.04642* (2025).

[28] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740. doi:10.1109/TSE.2016.2521368

[29] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] https://arxiv.org/abs/2308.08155

[30] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).

[31] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024. TravelPlanner: A Benchmark for Real-World Planning with Language Agents. arXiv:2402.01622 [cs.CL] https://arxiv.org/abs/2402.01622

[32] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[33] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2025. AgenTracer: Who Is Inducing Failure in the LLM Agentic Systems? arXiv:2509.03312 [cs.CL] https://arxiv.org/abs/2509.03312

[34] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, et al. 2025. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. *arXiv preprint arXiv:2505.00212* (2025).

[35] Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, et al. 2025. Where llm agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370* (2025).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009